

Continuous Cartesian Genetic Programming based representation for multi-objective neural architecture search

Cosijopii Garcia-Garcia^{*}, Alicia Morales-Reyes, Hugo Jair Escalante

Computer science, INAOE, Luis Enrique Erro #1, San Andres Cholula, 72840, Puebla, Mexico

ARTICLE INFO

Article history:

Received 29 March 2023

Received in revised form 3 August 2023

Accepted 24 August 2023

Available online 9 September 2023

Keywords:

Neural architecture search
Cartesian genetic programming
Convolutional neural network
Multi-objective optimization

ABSTRACT

We propose a novel neural architecture search (NAS) approach for the challenge of designing convolutional neural networks (CNNs) that achieve a good tradeoff between complexity and accuracy. We rely on Cartesian genetic programming (CGP) and integrated real-based and block-chained CNN representation, for optimization using multi-objective evolutionary algorithms (MOEAs) in the continuous domain. We introduce two variants, CGP-NASV1 and CGP-NASV2, which differ in the granularity of their respective search spaces. To evaluate the proposed algorithms, we utilized the non-dominated sorting genetic algorithm II (NSGA-II) on the CIFAR-10, CIFAR-100, and SVHN datasets. Additionally, we extended the empirical analysis while maintaining the same solution representation to assess other searching techniques such as differential evolution (DE), the multi-objective evolutionary algorithm based on decomposition (MOEA/D), and the *S metric selection* evolutionary multi-objective algorithm (SMS-EMOA). The experimental results demonstrate that our approach exhibits competitive classification performance and model complexity compared to state-of-the-art methods.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

Deep neural networks (DNNs), particularly convolutional and recurrent neural networks, have recently gained considerable popularity for approaching a wide variety of problems [1–4]. Regarding convolutional neural networks (CNNs), these are very effective computational models that have been thoroughly investigated in a wide range of image processing and computer vision-related tasks. This has been possible thanks to a number of factors including availability of large amounts of data, high-performance computing resources and research advances in the machine learning field [5].

CNNs are often organized according to complex topologies (architectures) whose design, including hyperparameter configuration, is done by expert users. While CNNs grow in complexity, manual configuration becomes time consuming and in some cases unfeasible [5]. Neural Architecture Search (NAS) is a field that aims at automating the design and configuration of CNN models [6]. NAS methods explore the space of CNN topologies looking for an architecture that meets certain criteria, for instance, achieving a minimum performance, or being *light* in terms of the number of parameters.

Nowadays, progress in NAS research has resulted in the identification of novel CNNs with favorable performance on

representative image classification datasets, attracting the scientific community's attention to this topic [7].

Evolutionary computation (EC) is a set of techniques inspired by the process of natural evolution to deal with complex optimization problems among others. The multi-objective approach deals with problems where objectives are in conflict and focuses on finding a set of solutions that represent a trade-off between them [8]. EC techniques have been successfully applied in the multi-objective optimization problems domain. In the context of NAS, the automated design of high-performance and low-complexity network architectures remains an open issue. This relevant problem can be formulated as one of multi-objective optimization [7,9–12].

The underlying hypothesis behind NAS is the fact that in real-world scenarios more computationally efficient and accurate CNN models are required. Therefore, objectives such as latency, number of operations, or power consumption are optimized in addition to the standard error or accuracy objectives [13]. In this paper, we introduce two multi-objective NAS methods based on Cartesian Genetic Programming (CGP), which aim to design CNN architectures for image classification. Our proposed CGP-NAS variants extend the original CGP-NAS [11] approach by considering a block-chain encoding. A first variant, called CGP-NASV1 is based on fixed CNN blocks to perform the NAS. While a second methodology, named CGP-NASV2, expands the search space to include the hyperparameters while relaxing the CGP-NASV1 fixed-blocks restriction. Our proposed NAS methods are evaluated in the context of image classification with CNNs.

^{*} Corresponding author.

E-mail addresses: cosijopii@inaoe.mx (C. Garcia-Garcia), a.morales@inaoe.mx (A. Morales-Reyes), hugojaire@inaoe.mx (H.J. Escalante).

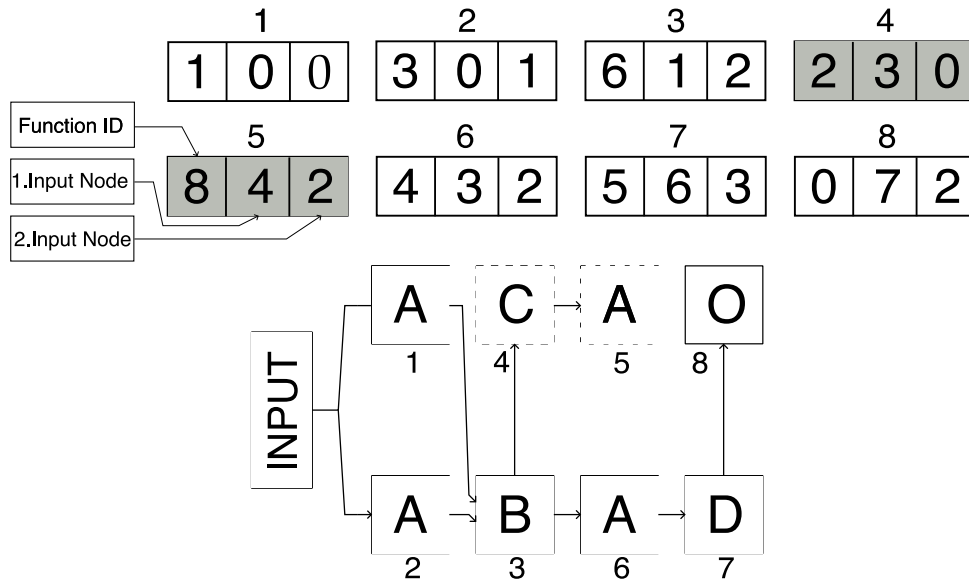


Fig. 1. CGP representation. Top: integer-based genotype representation, Bottom: mapping to the phenotype space as an acyclic graph. Vectors in gray represent no connection, thus these are inactive nodes in the acyclic graph.

Experimental results in the CIFAR-10, CIFAR-100 and SVHN datasets show that both proposals are competitive against several state-of-the-art references. The evolved CNNs architectures have a lower number of parameters as well as lower complexity measured in Multiply-Adds (MAdds) operations in comparison to the state-of-the-art and maintain a low classification error.

A preliminary version of CGP-NAS was recently introduced in our previous work [11]. This NAS solution considers an encoding-decoding process, first using CGP to represent CNN architectures and then converting the CGP integer-based encoding to a real-based representation. A continuous representation allows to use canonical multi-objective evolutionary algorithms (MOEAs) for the optimization process. In fact, NSGA-II was used as an optimizer for CGP-NAS in previous work [11].

This paper presents two novel evolutionary multi-objective NAS approaches based on our original CGP-NAS proposals. In this one, new features were added, such as the use of a high-level representation based on block-chain encoding, which allows for better control over the general shape of the architecture to be searched; also, new blocks of convolutional operations were added to the functions set; and finally, the representation was modified to have the feature of being able to evolve the hyperparameters together with the CNN architecture. These models achieve competitive, state-of-the-art performance. Both enhancements are described in detail and extensively evaluated. NSGA-II [14], MOEA/D [15] and SMS-EMOA [16] are considered optimizers in our experimental evaluations.

The motivation for this research is to address the challenges of multi-objective neural architecture search by exploring the use of a continuous representation based on CGP. We aim to demonstrate the advantages of continuous representations over discrete ones in optimizing neural architectures, particularly in multi-objective scenarios. Additionally, the research analyzes the behavior of multiple MOEAs to understand how different algorithms influence the quality and diversity of the solutions obtained. Furthermore, we explore the block-chained representation and its combination with the proposed continuous representation to reduce the complexity of the search space. By combining these representations, we seek to enhance the flexibility and adaptability of the search process. Overall, we aim to improve the efficiency and effectiveness of neural architecture search by posing the problem as a multi-objective optimization one while

having the benefits of a continuous solution representation and the possibility of searching with other standard MOEAs.

The remainder of this paper is organized as follows. Section 2 presents background information required to understand our contribution. Next, Section 3 reviews related work focusing on NAS methods based on evolutionary computation. Section 4 introduces our proposed CGP-NAS variants. Then, Section 5 reports the empirical assessment and Section 6 discusses the achieved results. Finally, Section 7 outlines conclusions and future work directions.

2. Background

In this section, we provide an overview of the fundamental concepts of Cartesian Genetic Programming (CGP) and solution representations for Neural Architecture Search (NAS). CGP is at the core of the proposed approach, thus a comprehensive introduction is provided. Additionally, we offer a broad overview of the various representations utilized in NAS. This context is vital to situate and contextualize our proposal within the broader landscape of NAS methodologies.

2.1. Cartesian genetic programming

Cartesian Genetic Programming (CGP) is a Genetic Programming (GP) methodology based on acyclic graphs for solution representation that allow forward connections. This offers a number of improvements with respect to standard GP, where a tree-based representation is used. CGP was initially conceived to evolve digital circuits. It was proposed by J.F. Miller [17] and is called cartesian because it represents solutions using a two-dimensional grid [18]. CGP shares important features with GP, such as the definition of a set of functions and their arity. CGP can also be applied to different areas, such as the automatic design of digital circuits, mathematical equations, and even artistic applications [17,18].

In the genotype space, a solution is mapped as an integer-based vector divided by segments that represent the function identifier (a predefined function from a set of functions that are the graph's nodes) and its connections. The size of each segment varies depending on the maximum arity of the represented function.

Fig. 1 shows a CGP solution representation example with maximum arity of two and its mapping to the phenotype space.

Another CGP characteristic is the inactive nodes that are not expressed in the phenotype, these are represented as gray sections and dotted-line squares in Fig. 1. This information is maintained in the genotype and is exploited during the evolutionary process, for example, a change of an important connection between two functions maps as a big change in its corresponding phenotype. In canonic CGP, the crossover operator is not used (but can be implemented); instead, only mutations occur as random changes in connections and nodes.

The CGP is set up on a fixed-size grid $N_r \times N_c$, with the number of connections between nodes determined by an l variable known as level-back. Normally, the number of inputs and outputs depends on the problem. CGP in its canonical version is combined with the $(1 + \lambda)$ evolutionary strategy algorithm. However, CGP can be adapted to other evolutionary search algorithms like NSGA-II or genetic algorithms [18].

2.2. Solutions representation in NAS

NAS automates the design of neural networks given a specific task; for this, a search space of architectures must be specified, as well as a set of input data [13,19]. The complexity of the search space is indicated by the architecture representation. Neural architecture representation in NAS influences the design of the evolutionary algorithm as well as the complexity of the search space, and therefore it is of vital importance to understand the different ways in which the NAS representation can be classified. Fig. 2 shows a general taxonomy of encoding representations with three main categories [20]: direct, indirect, and hybrid encodings.

Direct encoding is based on graphs, trees, and block chains. Evolutionary operators struggle to adapt to this kind of encoding. A decoding mechanism is not necessary.

Indirect encoding relies on binary or integer representations; these are simpler and easy to adapt to standard evolutionary operators. However, they are less flexible and usually reduce the search space.

Hybrid Encoding is a combination of direct and indirect encoding. In the literature, a two-level block-chained representation is used on the first level and binary encoding on the second level. In the context of CGP-NAS [11], CGP is used as a first-level encoding and real-based representation on a second level.

3. Related work

The automated design of convolutional neural networks has been expanded by using neural architecture search. In a number of cases, the architectures discovered by NAS perform better than expert-tuned ones [7]. Commonly, NAS has been approached as a single-objective optimization problem, trying to minimize the classification error of architectures. However, several approaches have posed the NAS problem as a multi-objective optimization problem, targeting conflicting objectives such as the classification error, architecture complexity, and learnable parameters [10]. The following sections present an analysis of different single-objective and multi-objective NAS methodologies based on EC. Finishing with a discussion of different ways to represent the solutions.

3.1. Single versus multi-objective NAS methods

NAS aims to automate the design process of neural network architectures. Several searching mechanisms have been used, such as evolutionary computation (EC), reinforcement learning (RL) [21–23], as well as relaxation methods [24–26]. Normally, NAS is divided into three main components: search space, search strategy, and the strategy to estimate the performance of each

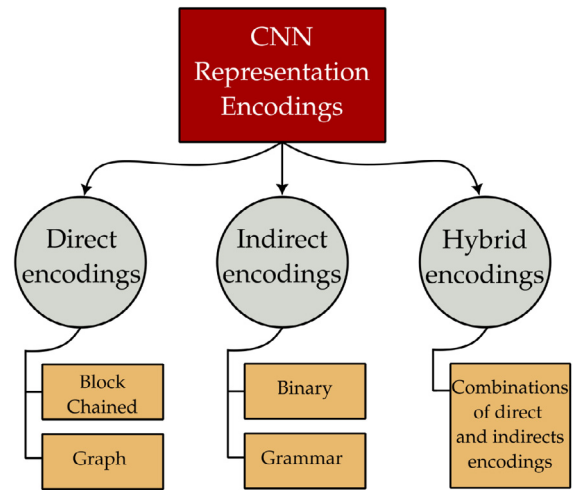


Fig. 2. Representation types taxonomy to encode CNNs.

architecture. The first two components are linked to the representation and the search method; the strategy is most commonly used to estimate the performance of each architecture during the evolutionary search to obtain a training and validation subsets from the complete dataset, and therefore a performance estimate of the designed architecture [6]. Other performance estimation mechanisms, such as those based on Bayesian programming or surrogate models [7,9,10], have been used to reduce the total number of evaluations and thus the searching time.

Most evolutionary single-objective NAS methods aim at decreasing the classification error rate, for example, Xie et al. introduced GeNet [27], one of the first NAS models, where a straightforward model based on binary-fixed representation and a genetic algorithm was proposed. After GeNet, multiple proposals were introduced such as CGP-CNN [28] using Cartesian genetic programming (CGP) and a block-based design, AE-CNN [29] with the same idea of a block-based search but using a variable-length representation. DSGE-CNN [30] used the evolutionary grammar-based genetic programming algorithm with a block-based approach including the hyperparameters. This variant of Genetic Programming (GP), has become relevant because of the flexibility of the representation [31].

Tobari [32] presented a new NAS algorithm that uses CGP, like CGP-CNN, which is a block-based algorithm that uses evolutionary strategies for the search. Its main contribution is to add a CGP crossover method based on the Multiple Sequence Alignment algorithm (MSA) inspired by the alignment of amino acid sequences, showing improvement in comparison to the canonical CGP.

Bakhshi [33] proposed to add the hyperparameters to the search, all of these proposals have in common the use of pre-defined blocks. CIFAR-10 and CIFAR-100 datasets were used for evaluation. The use of a variable-length representation similar to CGP-CNN, AE-CNN, DSGE-CNN, Tobari et al.'s and Bakshi et al.'s solutions showed an improvement compared to initial proposals such as GeNet.

The automatic search for improved CNN architectures, in terms of better performance metrics or reduced complexity, inherently poses a multi-objective optimization problem. Including more architecture's elements within the search, such as complexity (FLOPs or MAdds) or the number of learnable parameters [12] prompted the area of Multi-objective Neural Architecture Search. NEMO [34] was among the first proposals with a multi-objective approach. Later, NSGA-Net [9] stood out for its performance

in finding CNN architectures with low error and complexity, followed by NSGA-NetV1 [7] and NSGA-NetV2 [10].

NSGA-Net uses binary representation in addition to a block-based approach with spatial reduction steps (pooling). On the other hand, NSGA-NetV1 uses the cell-based representation approach in which stacked blocks have internal connections between predefined blocks for convolution and pooling operations [35]. NSGA-Net extends the number of operations that can be performed, adding functions like dilated convolution and separable convolution. Thus, NSGA-NetV1 uses graph-based encoding to represent solutions and both crossover and mutation operations. Finally, NSGA-NetV2 extends NSGA-NetV1 to a surrogate approach using integer-based encoding.

MOCNN encodes data using an Internet Protocol Address-based approach to define the number of parameters within a fixed DenseNet architecture and uses the Particle Swarm Optimization (PSO) algorithm for the architectures search [36]. In MOGI-NET [37], the encoding was inspired by NSGA-Net, the objectives are to minimize the classification error and the network parameters. It uses weight inheritance to reduce the number of epochs required to train the network, thus the offspring generated required less training.

EEEE-NET [12] used a cell-based representation similar to NSGA-NetV1. It is based on a mechanism of early exit population initialization; a maximum threshold of learnable parameters is defined; if a randomly generated solution has a low number of parameters compared to the maximum threshold, it is added to the initial population; otherwise, another solution that meets this threshold is chosen. Authors intended, with this condition, to reduce the training time of each architecture, which impacts the total search time.

LF-MOGP [38] is a multi-objective proposal that uses a representation based on CGP and has as a major contribution the use of a leader-follower evolution mechanism. With this, during the evolutionary process, it keeps in an external archive the best solutions to generate diversity during the search. In its proposed evolutionary algorithm, it highlights the use of crossover and mutation, since in CGP, normally the crossover operators are not used. The objectives to be pursued in this work are the accuracy of the model as well as the complexity measured in the number of parameters.

EvoApproxNAS [13] is a method based on CGP that targets three objectives in conflict: the power consumption of the processing platform, network parameters, and classification error. EvoApproxNAS increases the number of blocks for CGP by adding bottleneck residuals and inverted residual blocks.

Finally CGP-NAS [11] is a multi-objective approach based on a Cartesian genetic programming. It minimizes the architecture complexity while reducing the classification error. CGP-NAS also combines CGP with a real-based encoding-decoding method; thus, solutions are manipulated in the continuous domain, and consequently, the classical multi-objective evolutionary algorithms can be used, in the case of this work NSGA-II.

We use CGP-NAS as a basis for this work, we extended several points, such as the representation and the number of blocks in the function set, as well as the addition of hyperparameters to the search.

3.2. Discussion

Most of the methods discussed so far have either proposed new representations or adapted modifications to previous ones. Accordingly, crossover and mutation operators are also being developed. Having a variable-length encoding increased the NAS performance because the search space coverage increased. On the other hand, proposals that use information to estimate new

solutions without training can significantly reduce the number of evaluations and have the advantage of using fewer GPU days. We can also note the limited use of bio-inspired methods such as PSO and the dominance of multiobjective evolutionary algorithms such as NSGA-II.

Our proposal implements a variable-length representation using CGP, that is also converted to a real-based encoding, thus avoiding any adaptation of the crossover and mutation operations. This speeds up CGP-NAS implementation through the ability of searching using different MOEAS as well as the possibility of having multiple types of crossover and mutation operators. Finally a summary of the methods presented here is presented in Table 1.

4. Continuous cartesian genetic programming based representation for multi-objective NAS

The automated configuration of CNN architectures has been mostly driven by methods that aim to optimize a single objective, commonly, accuracy. While effective architectures can be obtained by optimizing model's performance, such an approach has a number of limitations, the most important is perhaps the propensity of solutions to overfitting because model complexity is not restricted. Likewise, model complexity is tied with efficiency, hence, low complexity models should be preferred. In this work, we propose the design of CNN architectures that simultaneously maximize model's accuracy and minimize its complexity. The goal is to restrict the capacity of the model with the aim of obtaining accurate models based on architectures of moderated complexity. Compared to alternative solutions, ours is based on a CGP representation that allows us to operate in the real domain. This enables the usage of off-the-shell multi-optimization techniques.

The approached problem can be formulated as one of multi-objective optimization as follows:

$$\begin{aligned} \text{Minimize } \mathbf{F}(x) &= f_1(x), w^*(x), f_2(x))^T \\ \text{subject to: } w^*(x) &\in \operatorname{argmin} \mathcal{L}(w; x) \end{aligned}$$

Where f_1 is an objective associated to the classification error of the CNN architecture as defined by parameters w^* and f_2 is an objective associated to model complexity measured, in agreement with previous studies, with MAdds, as they express the total number of operations performed in each architecture. One should note that MAdds are a guideline for certain implementation scenarios, such as under mobile settings, where the complexity must be less than or equal to 600 MAdds [10,39]. Please note that in order to obtain an estimate of classification error it is necessary to optimize the weights w of the CNN architecture, so x (solution) depends on this optimization where normally some training algorithm such as stochastic gradient descent (SDG) is used.

Our approach to this problem relies on evolutionary algorithms, with a set of individuals (a population of CNN architectures) being generated randomly, to be subsequently selected according to their fitness. This subset of solutions goes through a process of crossover and mutation to evolve in new offspring. Finally, the new offspring and the current population are rearranged so that only the solutions with better fitness pass to the next generation.

Two variants of our method are proposed. We first introduce a CGP-NASV1, a variant using a block-chained approach, see Fig. 3. In each "Normal" block, an internal CGP representation itself handles the connections and functions, in the "Reduction" blocks, a spatial reduction is applied, in our case, a pooling block is fixed. During the optimization process, this representation is re-encoded to a real-valued representation. Therefore,

Table 1
Summary of methods their objectives, datasets and encoding.

Method	Objectives	Datasets	Encoding
GeNet [27]	Accuracy	CIFAR-10, ImageNet	Binary
CGP-CNN [28]	Accuracy	CIFAR-10, CIFAR-100	CGP
Torabi et al. [32]	Accuracy	CIFAR-10, CIFAR-100	CGP
AE-CNN [29]	Accuracy	CIFAR-10, CIFAR-100	Block-Chained
DSGE-CNN [30]	Accuracy	CIFAR-10	Grammar
Bakhshi et al. [33]	Accuracy	CIFAR-10, CIFAR-100, SVHN	Integer
NEMO [34]	Accuracy, Inference Speed	MNIST, CIFAR-10	Integer
NSGA-Net [9]	Accuracy, MAdds	CIFAR-10, CIFAR-100	Binary
NSGA-NetV1 [7]	Accuracy, FLOPs	CIFAR-10, CIFAR-100, ImageNet	Block-Chained, Graph
NSGA-NetV2 [39]	Accuracy, MAdds	ImageNet, CIFAR-10, CIFAR-100	Block-Chained, Integer
MOCNN [36]	Accuracy, FLOPs	CIFAR-10	Binary
MOGI-NET [37]	Accuracy, Parameters	CIFAR-10, CIFAR-100	Binary
CGP-NAS [11]	Accuracy, MAdds	CIFAR-10, CIFAR-100	CGP, Real
LF-MOGP [38]	Accuracy, parameters	CIFAR-10, CIFAR-100, Fashion, MNIST variants	CGP
EvoApproxNAS [13]	Accuracy, parameters, Energy	CIFAR-10, SVHN	CGP

the full MOEAs searching potential within continuous domain is exploited. After every generation, solutions are re-encoded to CGP and evaluated. In a second variant of CGP-NAS, we included the hyperparameters of the CNNs in the CGP representation, increasing the flexibility of model configurations that can be found, at the expense of increasing the search space. The remainder of this section describes both variants in detail, we first describe the associated search space and then present the CGP representations for both variants, then the considered evolutionary algorithms are presented.

4.1. Search space

CGP works on a set of functions which implicitly define the search space, see Table 2. We considered 11 functions for CGP-NASV1, where we need to make explicit all of the possible combinations in the number of channels and kernels. In CGP-NASV1, the representation is limited to three “Normal” blocks, where each block has a variable channel size. The first block handles functions with 32 or 64 output channels, the second block 64 or 128 and the third block 128 or 256, the larger the number of “Normal” blocks, the larger the number of output channels. Blocks with no variants are marked with “-”. The total number of functions-blocks in the function set is calculated as the possible combinations of output channels C and Kernels k , which are in total 68 functions-blocks.

In CGP-NASV2, the hyperparameters are encoded explicitly in the CGP representation, therefore it is not necessary to explicitly list all of the combinations, hence the total number of functions are significantly reduced. Adding the hyperparameters to the representation increases the size of the vector of decision variables and makes the problem slightly more complex, but on the other hand, it makes the representation more flexible as well as reduces the number of functions.

In this work, we extended the number of blocks used in our previous work, adding more complex ones such as FusedMB-Conv [40] from EfficientNetV2, MBConv [41] from MobileNetV2, and Bottleneck blocks [42], as well as blocks that have proven their effectiveness related work [7,9] such as Dilated Convolution, Deep-wise separable Convolution and 1×7 then 7×1 Convolution. Moreover, the identity block is also included helping to generate more flexible architectures with no added computational cost.

4.2. CGP-NASV1 solution representation

CGP-NASV1 uses the block-chained encoding at the top level, it defines an template with some layers as shown in Fig. 4. This representation connects blocks linearly, including those with

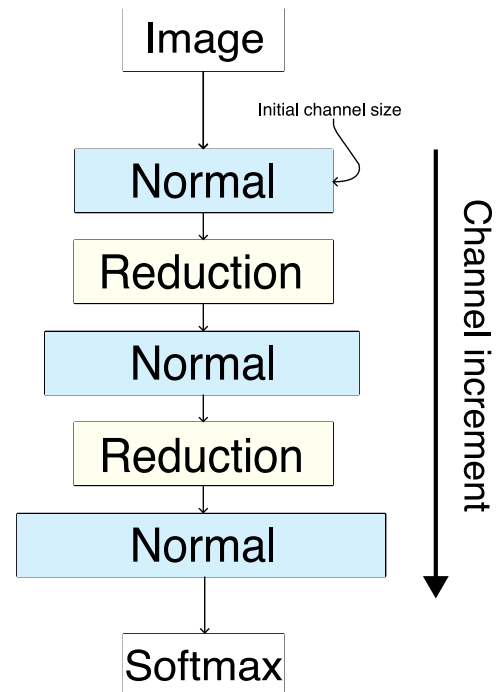


Fig. 3. The general scheme of the representation based on chained blocks; each block internally focuses on special operations; in each normal block, for example, convolution operations are performed; on the other hand, in the reduction blocks, methods such as pooling are applied to spatially reduce the feature maps. As the number of blocks increases, the number of channels will gradually increase.

specific tasks, in CGP-NASV1 the spatial reduction is performed by the pooling layers [7].

At the top-level in the block-chained, CGP-NASV1 places a CGP within a “Normal” block while the reduction blocks implements a max pooling layer. In all pooling blocks, the pooling size is fixed as a 2×2 kernel and a stride of 2. Fig. 4 shows an example, after the last block a global average pooling and a fully connected layer are added. Similar guidelines were found in the state of the art review [7,13].

Fig. 5 shows the final representation as an integer-based vector, where each square that encapsulates the function represents a “Normal” block with its own CGP. Each CGP is configured independently, for example, its own function set and size. In the figure, the red blocks contain the ID of a function.

Table 2
Functions set with corresponding variations and arity.

Block type	Symbol	CGP-NASV1	CGP-NASV2	Arity
ConvBlock	$CB(C, k)$	$C \in \{32, 64, 128, 256\}$ $k \in \{1 \times 1, 3 \times 3, 5 \times 5, 7 \times 7\}$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
ResBlock	$RB(C, k)$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5, 7 \times 7\}$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
Bottleneck	$BN(C, k)$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5, 7 \times 7\}$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
FusedMBconv	$FBC(C, k)$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3\}$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
MBconv	$MBC(C, k)$	Not used in CGP-NASV1	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
SepConv	$SC(C, k)$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5, 7 \times 7\}$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
DiConv	$DC(C, k)$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	$C \in \{32, 64, 128, 256\}$ $k \in \{3 \times 3, 5 \times 5\}$	1
Identity	I	-	-	1
$C1 \times 7-7 \times 1$	$C17$	-	-	1
Summation	Sum	-	-	2
Concatenation	Concat	-	-	2

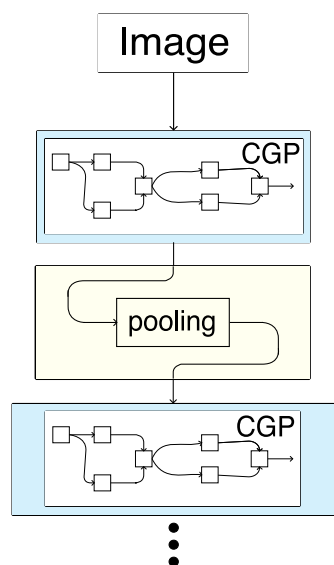


Fig. 4. Block-chained representation.

We maintained the original idea from CGP-NAS of having two levels for representation [11]. In CGP-NASV1, each “Normal” block from the block-chained representation holds a CGP and it is represented as an integer vector.

At the second level of representation, the integer vector is encoded to a real-based vector. Eq. (1) defines a range considering $func_k$ as the current function identifier and $func_{total}$ as the total number of functions in the function set. A uniform random number is generated to represent a function within this range in the real domain.

$$rfunc_k \in \left[\frac{func_k}{func_{total}}, \frac{func_k + 1}{func_{total}} \right] \quad (1)$$

Eq. (2) defines a range for the function inputs in the real domain to map connections for that node. This operation is applied to all connections. An input value (*nodeinput*) and its node number (*nodeTerm*) are used to calculate this range.

$$rinput_j \in \left[\frac{nodeinput_j}{nodeTerm}, \frac{nodeinput_j + 1}{nodeTerm} \right] \quad (2)$$

$$func_k = |gene_i \times func_{total}| \quad (3)$$

$$input_i = |gene_i \times NodeTerm| \quad (4)$$

In order to decode solutions from the real to the integer domain, Eq. (3) obtains the function identifier by multiplying the gene value by the total number of functions. Moreover, Eq. (4) obtains the value of every connection by multiplying the gene value with the node number. The equations above are based on Clegg's work [43].

CGP-NASV1 represents CNN architectures in a divide and conquer approach. The use of a block-chained schema in synergy with CGP allows more control over the final architecture,

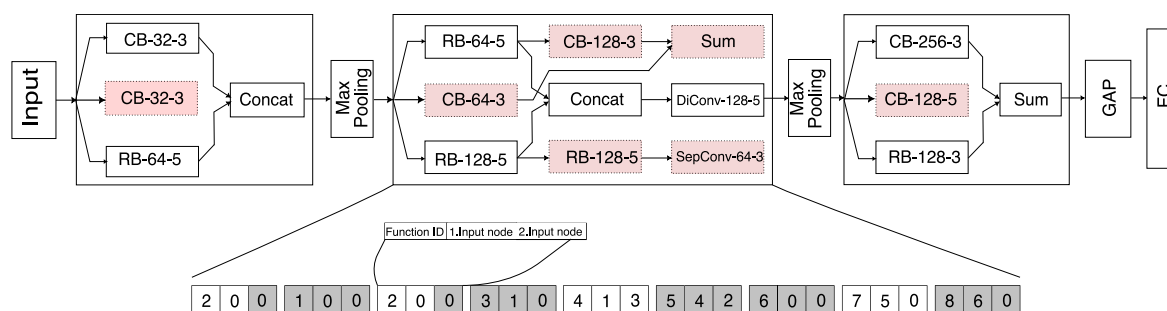


Fig. 5. CGP-NASV1 using the block-chained representation.

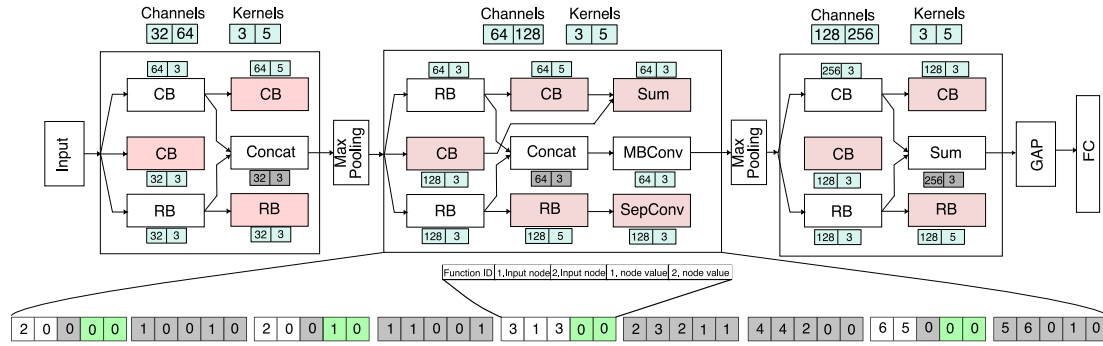


Fig. 6. CGP-NASV2 block-chained representation with the hyperparameters directly encoded.



Fig. 7. A CGP-NASV2 solution represented as an integer vector and its equivalent real-based vector with the added hyperparameters in the green positions.

Our initial proposal CGP-NAS defined the size of the grid on which the complete neural architecture would be search. However, dealing with the search of an entire architecture at once can lead to very large and complex ones. This can be avoided, by dividing the overall architecture in more manageable stages.

4.3. CGP-NASV2 representation

CGP-NASV2 proposal integrates the hyperparameters within the solutions representation of CGP-NASV1. Thus, they are also evolved and optimized during the search. Changes in CGP-NASV2 are at the block-chained “low level” encoding. Therefore, there is a reduction in the number of functions within the function set. However, adding the hyperparameters to the search, necessarily increases the size of the integer vectors (and also the real-based vectors) to which the block-chained solutions are encoded.

In a deep neural architecture, there are hyperparameters such as the number of channels or filters and the kernel size. In CGP, these are explicitly included in the functions set if not considered within the solutions representation and in the evolutionary search. Therefore, a large function set must be defined in order to include all their possible combinations. Moreover, a large function set directly increases the CGP grid size to hold them uniformly.

In CGP-NASV2, we add the hyperparameters explicitly to the vector representing the CGP. This reduces the number of functions since only the standard functions remain and not their hyperparameters configuration.

In Fig. 6, a possible solution in CGP-NASV2 is shown. The number of channels and kernel size are defined as parameters associated to each block-chained. The idea is to associate these parameters as weights to each CGP node. Green positions at the integer vector encoding in Fig. 6 represent the assigned hyperparameters within their corresponding CGP block.

Fig. 7 shows an example of a CGP-NASV2 solution representation as an integer vector with hyperparameters. The entire vector is modified once the hyperparameters are explicitly considered by the CGP. The first three positions (white and gray positions) represent the id of the function and the input connections nodes. The green positions show two hyperparameters, one corresponding to the #channels and the other to the #kernels. For example, in the

integer vector representation from Fig. 6, 0 in position 4 means that the value of the channel is 64 and 1 in position 5 means a kernel of 5×5 .

To convert the integer vector to the real-based vector, the same mechanism as the one used in CGP-NASV1 is applied adding only Eqs. (5) and (6) to encode and decode the hyperparameters.

$$rHyp_k \in \left[\frac{Hyp_k}{Hyp_{total}}, \frac{Hyp_k + 1}{Hyp_{total}} \right] \quad (5)$$

$$Hyp_j = \lfloor gene_i \times Hyp_{total} \rfloor \quad (6)$$

The difference between both proposed variants is that in CGP-V1 the function set is restricted to explicit instantiations of the function set. While in CGP-NASV2 this restriction does not hold, the algorithm itself has to figure out what functions what hyperparameters (filter size, channels) to use for building solutions. This provides more flexibility at the cost of increasing the search space.

4.4. Evolutionary algorithm

Working in a real-valued representations allows us to use any MOEA operating in such domain. Therefore, we initially considered the NSGA-II MOEA as optimizer, as this is one of the most used and effective MOEA. Fig. 8 shows a general schema of this widely known MOEA. Because, both CGP variants represent solutions in the continuous domain to perform the search, the full properties of NSGA-II are maintained. The only significant change is for the crossover and mutation operations. Both were adapted for CGP-NASV1 and CGP-NASV2 due to the use of block-chained representation.

Each individual, represented as a real-based vector, is divided in three sub-vectors, each one encodes a CGP “Normal” block. Fig. 9 shows how the crossover and mutation operations are applied. The crossover operation is performed between sub-vectors at the same overall position within the solution full vector. Also, the mutation operation is applied to the offspring of every sub-vector independently. It is important to emphasize that each operation is carried out independently at a sub-vector level. In the figure’s example, three crossover and mutation operations are

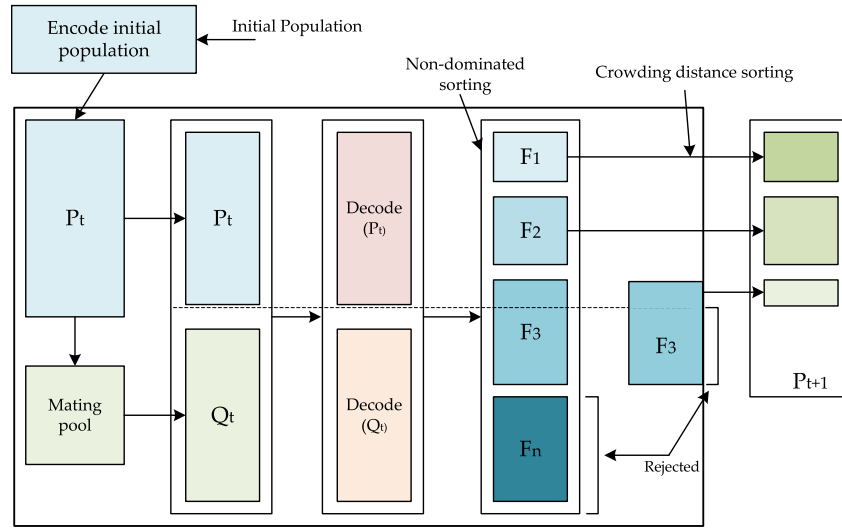


Fig. 8. NSGA-II general schema with encoding and decoding steps. NSGA-II generates an initial population P_t , after mating an offspring population Q_t is obtained. Q_t is decoded for evaluation, and therefore for optimization considering the non-dominance criterion.

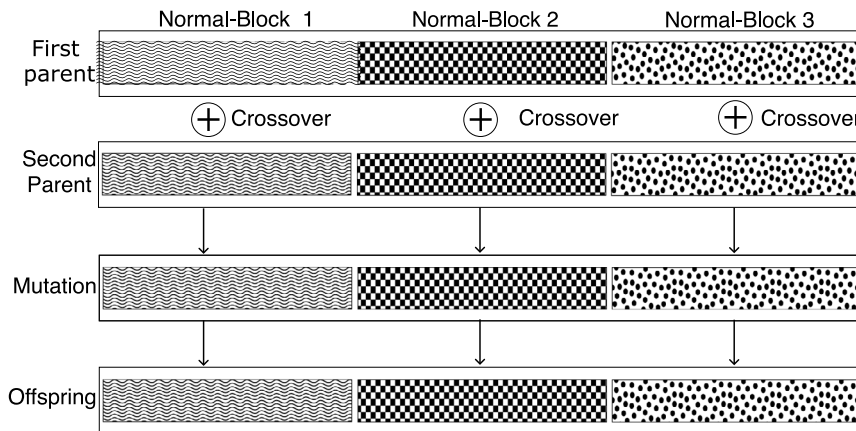


Fig. 9. Each sub-vector represent a CGP “normal” block. Crossover applies independently between sub-vectors at the same overall position. Mutation works on the sub-vectors offspring.

carried out for each sub-vector's offspring (pairs of CGP “Normal” blocks). These together represent the complete neural architecture to assess. The crossover and mutation operators are defined by those commonly used in the NSGA-II; the simulated binary crossover (SBX) and the polynomial mutation (PM).

4.5. Fitness function

As previously mentioned in this paper we aim to simultaneously optimize two objectives with both variants of CGP-NAS, these are: accuracy and the complexity as measured in MAdds. Where we want to maximize accuracy and minimize the complexity. The accuracy estimate is obtained by evaluating the validation set, which is a random subset of the training set, this way of obtaining the accuracy is called partial training performance estimation strategy [6].

We calculate the classification error after the accuracy using the following expression $1 - \text{Accuracy}$, in order to minimize both objectives. A CNN architecture relies on the convolution operation which mainly performs multiplications and addition

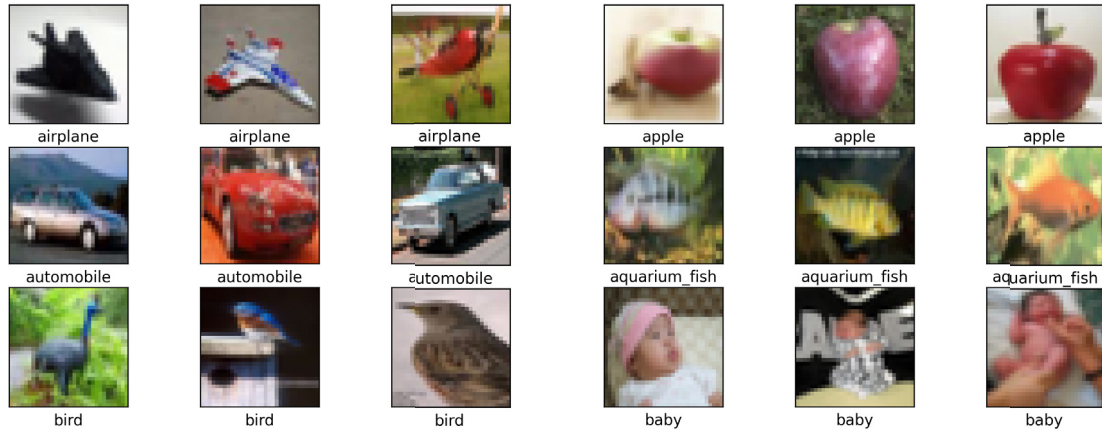
operations. Therefore, the complexity is calculated as the total number of multiplication-adds operations performed by the CNN. The complexity is calculated as the sum of multiplication and addition operations (MAdds) performed by the CNN architecture and is the second objective to minimize.

5. Experimental framework

In order to assess both proposed approaches, CGP-NASV1 and CGP-NASV2, first a direct performance comparison between the best evolved architecture based on the achieved accuracy as the performance metric is carried out. After, a multiple-criteria decision analysis is performed to evaluate those evolved CNN models that better achieve a more balanced trade-off performance considering both objectives, accuracy and complexity. In this section, we present the experimental settings for both experiments as well as the considered datasets. In the next section, we discuss the obtained results and thoroughly compared them against the original CGP-NAS [11] proposal and several state of the art approaches.

Table 3
CGP-NASV1 and CGP-NASV2 parameters.

Parameters	CGP-NASV1	CGP-NASV2
Rows	5	10
Columns	25	4
Level-Back	1	1
Mutation probability	$P_m = 0.3$	
Crossover probability	$P_c = 0.9$, distribution index for simulated binary crossover	
	$D_{sc} = 20$.	
Population	24	24
Generations	30	30



(a) CIFAR-10

(b) CIFAR-100



(c) SVHN

Fig. 10. Examples from the CIFAR-10, CIFAR-100 and SVHN datasets, the image in each row belongs to the same class.

Table 3 shows CGP-NASV1 and CGP-NASV2 configuration in terms of the CGP and the evolutionary algorithm. The fundamental modification in CGP-NASV2 is having the hyperparameters directly represented within solutions and therefore a reduction in the grid size is also required due to the smaller functions set. In contrast, CGP-NASV1 requires a larger CGP grid size due to the bigger functions set associated to express all possible hyperparameters combination.

For evaluation of the proposed algorithmic approaches, three datasets for image classification were used: the CIFAR-100 [44],

the CIFAR-10 [44] and SVHN [45]. CIFAR-100 is a 100 classes image collection, while CIFAR-10 is a 10 classes, each dataset consists of 50,000 images for training and 10,000 for testing, both datasets contain 32×32 pixels color images. Fig. 10 shows several image samples of these datasets. The SVHN dataset consists of 99,289 images, of which 73,257 for training and 26,032 for testing with 32×32 color images.

During the evolutionary search, solutions were evaluated using a training set randomly divided at a ratio of 80% for training and 20% for validation for all datasets used. For the final

evaluation of the selected solutions from the Pareto front, the entire training dataset is used to learn the parameters of the selected models, which are then evaluated in the test partition.

We use the following mechanisms for training the CNN architecture. The Stochastic Gradient Descent (SGD) was used, as an optimizer the cosine annealing learning rate schedule. The initial learning rate was set to 0.025, while the momentum was set to 0.9 and the weight decay to 0.0005. A 128 batch size was determined, in addition, 36 training epochs were executed during evolutionary search. Training and testing data were pre-processed with a 4 pixel-mean subtraction padding on each size and randomly cropped with a 32×32 patch or its horizontally flipped image. Final solutions were re-trained for 600 epochs and the cutout preprocessing technique was applied, the batch size was set to 96. Finally, an auxiliary head classifier [9] is used to improve the training process, concatenated after the second reduction block, and the loss of this auxiliary head classifier is multiplied by a constant value of 0.4 and added to the loss of the original architecture. All experiments were executed on a Supercomputing Node with: 2 Intel Xeon E5-2650 v4 @ 2.20 GHz processors, 8 Nvidia GTX 1080 Ti GPU cards and 128 GB of RAM, running on Centos 7 OS.

6. Results analysis

We designed three experimental setups in order to evaluate the proposed algorithmic approaches. The difference between the first two experiments is the way the final solution is selected from the achieved Pareto front. In the first experiment, the final solution is determined by the smallest classification error, similar criterion is followed by most state of the art works. Therefore, we selected the evolved CNN presenting the lower classification error. After, this solution is executed 10 times for a fair comparison between state of the art solutions, our original proposal CGP-NAS [11], and the other algorithmic approaches herein discussed in three datasets: CIFAR-10, CIFAR-100 and SVHN.

In the second experiment, CGP-NASV1 and CGP-NASV2 were compared through a multiple-criteria decision-making (MCDM) method to find the solution with the best Pareto front trade-off in the CIFAR-10 and CIFAR-100 datasets. In the third experiment, CGP-NASV2 is assessed while implementing other MOEAs and searching operations. In order to verify the representation developed, three different MOEAs were used: NSGA-II with the differential evolution crossover operator, MOEA/D, and SMS-EMOA using the CIFAR-100 dataset for comparison. Finally, we compare both proposed approaches versus the State of the Art works using the three datasets considered and discuss how the proposed solutions representation as well as the search mechanisms impact the final CGP-NASV1 and CGP-NASV2 performances.

6.1. Effectiveness of searching for the hyperparameters

In the first experiment, we empirically assessed the effectiveness of including the hyperparameters in the evolutionary search. Thus, we compared CGP-NASV1 versus CGP-NASV2 on the CIFAR-10 and CIFAR-100 datasets. The best evolved architectures in terms of accuracy from 10 independent runs were evaluated. We also added CGP-NAS as a baseline for comparison because it uses the same mechanism for selecting solutions from the Pareto front.

Table 4 shows the performance effect of evolving the hyperparameters. CGP-NASV2 reduces the error by half percentile point in comparison to CGP-NASV1 and in more than one percentile point with respect to the baseline approach CGP-NAS. This reduction is also observed for the standard deviation corresponding to the 10 experimental samples, demonstrating CGP-NASV2 is robust in comparison to the other approaches.

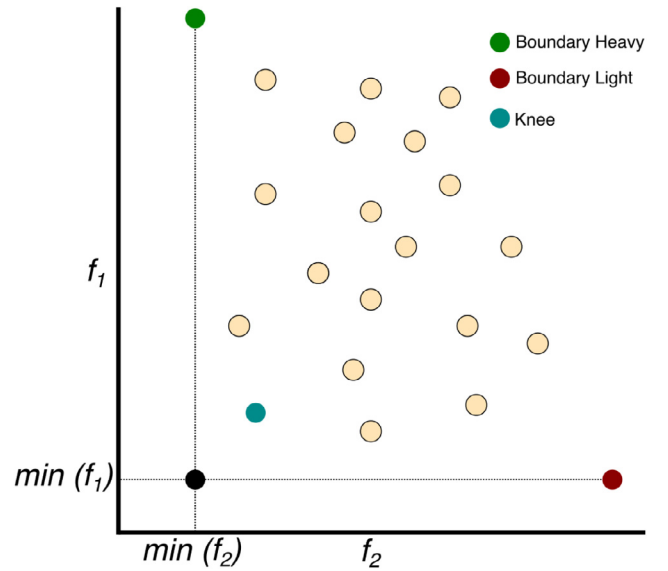


Fig. 11. Knee and boundary selection method. Measuring the distance between the intersections of the two best extreme solutions ensures the best trade-off solution is obtained.

We can also notice that the complexity measured in MAdds achieved by CGP-NASV2 is lower than that obtained by CGP-NASV1 for the CIFAR-10 dataset. On the other hand, both have similar values in MAdds terms for the CIFAR-100 dataset. It is worth mentioned that, the number of parameters is not one of objectives to optimize, therefore a significant variation among results is observed. Selecting the best evolved architecture by accuracy, easily shows that CGP-NASV2 achieves the best results in terms of the classification error. On contrast, the complexity objective is negatively affected by having also a higher standard deviation.

6.2. Best trade-off solution via multiple-criteria decision analysis

CGP-NASV1 and CGP-NASV2 are multi-objective proposals, therefore selecting the best solution in terms of accuracy from the Pareto front is not an efficient criterion. When evolving neural architectures as a multi-objective optimization problem, the best solution would represent a trade-off between objectives. Therefore, in the proposed algorithmic approaches, a trade-off between the classification error and the complexity of the architecture in terms of Madds. The Multi-criteria decision making (MCDM) was used to select the best solution from the Pareto front, in particular, the *Knee and Boundary Selection* method [46] has been explored for the empirical analysis.

This method obtains two solutions, called boundary heavy and boundary light, corresponding to the solution with the lowest value per objective. After, it calculates the solution closest to the intersection of both solutions (heavy and light), the obtained solution is called the Knee and represents the solution with the best trade-off. Fig. 11 shows how the knee and boundary selection works.

Table 5 shows the results achieved after applying the MCDM analysis to compare CGP-NASV1 and CGP-NASV2 in both CIFAR-10 and CIFAR-100 datasets.

The differences that we found with this selection method are mostly solutions with a lower number of parameters as well as MAdds. Although the classification error increased in comparison to that obtained by selecting the solution with the lowest classification error, the difference is relatively minor. Averaging 2% and

Table 4Parameters and MAdds expressed in millions (1×10^6)

	CIFAR10			CIFAR100		
	Error rate %	Parameters	MAdds	Error rate %	Parameters	MAdds
CGP-NAS [11]	4.86 (5.42 $\pm$ 0.46)	1.40 (2.52 + 0.90)	388.71 (1167.13+ 477.11)	24.23 (26.41 $\pm$ 1.41)	5.43 (5.89 + 2.75)	1581.93 (1229.11 + 782)
CGP-NASV1	4.23 (4.73 $\pm$ 0.44)	8.47 (8.74 $\pm$ 3.27)	1255.93 (1161.09 $\pm$ 373.78)	21.76 (24.20 $\pm$ 1.8)	3.6 (7.25 $\pm$ 3.24)	791.85 (792.02 $\pm$ 342.6)
CGP-NASV2	3.70 (4.07 $\pm$ 0.17)	4.04 (5.82 $\pm$ 2.70)	636.32 (818.61 $\pm$ 372.62)	20.63 (22.49 $\pm$ 1.04)	5.9 (6.50 $\pm$ 1.7)	827 (850.74 $\pm$ 476.08)

Table 5Trade-off knee solutions from the Pareto front, parameters and MAdds are expressed in millions (1×10^6).

	CIFAR10			CIFAR100		
	Error rate %	Parameters	MAdds	Error rate %	Parameters	MAdds
CGP-NASV1	5.66 (7.33 $\pm$ 1.55)	0.41 (8.74 $\pm$ 0.13)	70.13 (52.01 $\pm$ 21.09)	31.47 (33.34 $\pm$ 1.6)	1.06 (0.82 $\pm$ 1.12)	45.70 (30.81 $\pm$ 9.7)
CGP-NASV2	4.85 (5.59$\pm$0.5)	0.78 (0.71 $\pm$ 0.31)	53.99 (79.44 $\pm$ 31.96)	23.57 (28.23 $\pm$ 2.20)	0.49 (0.53 $\pm$ 0.13)	66.66 (55.66 $\pm$ 19.14)

Table 6Comparison on CIFAR-10 dataset: Classification error rate, the number of parameters and Multiply-adds (MAdds) are expressed in millions (1×10^6), GPU-days and GPU Hardware.

Model	Error rate %	Params	MAdds	GPU-Days	GPU hardware
Human Design					
DenseNet ($k = 12$) (Huang et al. 2017) [47]	5.24	1.0	–	–	–
ResNet ($depth = 101$) (He et al. 2016) [48]	6.43	1.7	–	–	–
ResNet ($depth = 1202$) (He et al. 2016) [48]	7.93	10.2	–	–	–
VGG (Simonyan et al. 2014) [49]	6.66	20.04	–	–	–
Single Objective Approaches					
CGP-CNN(ConvSet) (Suganuma et al. 2020) [50]	5.92	1.50	–	8	Nvidia 1080Ti
CGP-CNN(ResSet) (Suganuma et al. 2020) [50]	5.01	3.52	–	14.7	Nvidia 1080Ti
Large-Scale Evolution (Real et al. 2017) [51]	5.4	5.4	–	2750	–
AE-CNN (Sun et al. 2020) [29]	4.3	2.0	–	27	Nvidia 1080 Ti
Genetic-CNN (Xie et al. 2017) [27]	7.1	–	–	17	–
(Torabi et al. 2022) [32]	5.69	1.96	–	–	NVIDIA Tesla k80
Multi-Objective Approaches					
NSGANet (Lu et al. 2020) [10]	3.85	3.3	1290	8	Nvidia 1080 Ti
NSGANetV1 (Lu et al. 2020) [10]	4.67	0.2	–	27	Nvidia 2080 Ti
MOCNN (Wang et al. 2021) [36]	4.49	–	–	24	Nvidia 1080 Ti
MOGIG-Net (Xue et al. 2021) [37]	4.67	0.2	–	14	–
EEEA-Net (Termritthikun et al. 2021) [12]	2.46	3.6	–	0.52	Nvidia RTX 2080 Ti
EvoApproxNAS (Pinos et al. 2022) [13]	6.80	1.11	458.2	8.8	NVIDIA Tesla V100-SXM2
LF-MOGP(Liu et al. 2022) [38]	4.13	1.07	–	10	NVIDIA GeForce 3090
CGP-NAS (Garcia-Garcia et al. 2022) [11]	4.86 (5.42 $\pm$ 0.46)	1.40	388.71	1.4	Nvidia Titan X
CGP-NASV2-Best solution	3.70 (4.07 $\pm$ 0.17)	4.04 (5.82 $\pm$ 2.70)	636.32 (818.61 $\pm$ 372.62)	11.54	Nvidia 1080Ti
CGP-NASV2-Knee solution	4.85 (5.59 $\pm$ 0.5)	0.78 (0.71 $\pm$ 0.31)	53.99 (79.44 $\pm$ 31.96)	11.54	Nvidia 1080Ti

5% increase on CIFAR-10 and CIFAR-100 respectively. However, the reduction in terms of MAdds from 818.61M to 79.44M for CIFAR-10 and from 850.74M to 55.66M for CIFAR-100 represents a substantial decrease in model complexity. Highlighting the ability to find solutions that balance both objectives. Additionally, our results indicate that incorporating the hyperparameters within the search process is beneficial, particularly in terms of reducing the classification error.

6.3. Comparison versus the state of the art

Tables 6 and 7 present a detailed comparison between the State of the Art works and the proposed algorithmic approaches CGP-NASV1 and CGP-NASV2 on the CIFAR-10 and CIFAR-100

datasets. A total of 4 human designs, 5 NAS single-objective and 7 multi-objective proposals are considered for comparison plus the original proposal CGP-NAS [11] is also included in both tables.

From previous empirical assessments, it was determined that CGP-NASV2 provided the best overall results when compared to CGP-NASV1 and their baseline CGP-NAS. It was also analyzed that selecting the best solution from the Pareto front in terms of classification error would negatively affect the resulting evolved architecture in its complexity. Thus, a method to select trading-off solutions for both conflicting objects known as the knee and boundary methods allowed the selection of an evolved architecture with more balanced performance metrics. In evolutionary algorithms particularly in the multi-objective

Table 7

Comparison on CIFAR-100 dataset: Classification error rate, the number of parameters and Multiply-adds (MAdds) are expressed in millions (1×10^6), GPU-days and GPU Hardware.

Model	Error rate %	Params	MAdds	GPU-Days	GPU hardware
Human Design					
DenseNet ($k = 12$) (Huang et al. 2017) [47]	24.42	1.0	–	–	–
ResNet ($depth = 101$) (He et al. 2016) [48]	25.16	1.7	–	–	–
ResNet ($depth = 1202$) (He et al. 2016) [48]	27.82	10.2	–	–	–
VGG (Simonyan et al. 2014) [49]	28.05	20.04	–	–	–
Single Objective Approaches					
CGP-CNN(ConvSet) (Suganuma et al. 2020) [50]	26.7	2.04	–	13	Nvidia 1080Ti
CGP-CNN(ResSet) (Suganuma et al. 2020) [50]	25.1	3.43	–	10.9	Nvidia 1080Ti
Large-Scale Evolution (Real et al. 2017) [51]	23.0	40.4	–	2750	–
AE-CNN (Sun et al. 2020) [29]	20.85	5.4	–	36	Nvidia 1080 Ti
Genetic-CNN (Xie et al. 2017) [27]	29.03	–	–	17	–
(Torabi et al. 2022) [32]	26.03	2.56	–	–	NVIDIA Tesla V100-SXM2
Multi-Objective Approaches					
NSGANetV1 (Lu et al. 2020) [10]	25.17	0.2	1290	27	Nvidia 2080 Ti
MOGIG-Net (Xue et al. 2021) [37]	24.71	0.7	–	14	–
EEEA-Net (Termritthikun et al. 2021) [12]	15.02	3.6	–	0.52	Nvidia RTX 2080 Ti
LF-MOGP(Liu et al. 2022) [38]	26.37	4.12	–	13	NVIDIA GeForce 3090
CGP-NAS (Garcia-Garcia et al. 2022) [11]	24.23 (26.41 $\pm$ 1.41)	5.43	1581	2.1	Nvidia Titan X
CGP-NASV2 - Best Solution	20.63 (22.49 $\pm$ 1.04)	5.9 (6.50 $\pm$ 1.7)	827 (850.74 $\pm$ 476.08)	11.28	Nvidia 1080Ti
CGP-NASV2 - Knee solution	23.57 (28.43 $\pm$ 2.20)	0.49 (0.53 $\pm$ 0.13)	66.66 (55.66 $\pm$ 19.14)	11.28	Nvidia 1080Ti

optimization domain, Pareto front solutions represent a trade-off between conflicting objectives.

Another way to measure the performance of the proposed algorithms is through the Hypervolume metric. It determines the solutions distribution at the Pareto front which means how well they are distributed throughout the objective function space. The Hypervolume metric measures the total area the Pareto front covers with respect to a reference point, we use the nadir point [52]. This metric is calculated after every generation and the larger the value, the better. In Fig. 12 four box plots are observed each one represents the Hypervolume average value for every generation after 10 execution samples for the CIFAR-10 and CIFAR-100 datasets.

The final Hypervolume for CGP-NASV1 on CIFAR-10 is **0.85(0.651 $\pm$ 0.147)**, and for CIFAR-100 it is **0.618(0.505 $\pm$ 0.08)**. On the other hand, the results for CGP-NASV2 on CIFAR-10 are **0.69(0.551 $\pm$ 0.10)**, and for CIFAR-100, it is CIFAR-100 **0.51(0.37 $\pm$ 0.07)**.

From these results and the previous figures, we can observe that in the CIFAR-10 dataset, CGP-NASV1 tends to have more evenly distributed data, as indicated by the length of the whiskers and the size of the boxes, indicating a higher degree of variability in the data between experiments. Conversely, CGP-NASV2 displays less variability in the data, resulting in a more concentrated distribution. On the CIFAR-100 dataset, we can see a similar trend where CGP-NASV1 tends to have more evenly distributed data, while CGP-NASV2 shows a more concentrated distribution and more similar solutions between experiments; however, in the final generations, outliers are generated, which can be caused by sudden and unpredictable changes in the Pareto front resulting from the random nature of the searching algorithm; CGP-NASV2 also influences this behavior because it adds hyperparameters to the search. Summarizing, one can conclude that CGP-NASV1 tends to favor exploration, while CGP-NASV2 favors exploitation at the end of the search processes.

The search process of CGP-NASV1 and CGP-NASV2 on the CIFAR-10 and CIFAR-100 datasets is illustrated in Figs. 13(a) and 13(b), respectively. The transparency of the circular marks in the figures denotes the relative early stages of the generations. The

Pareto front for each proposal is depicted by the corresponding lines. Initially, it can be noted that the solutions are primarily located in the upper left zone, indicating more complex solutions. As the generations progress, solutions are placed in areas further to the right. The region with the most optimal trade-off is located in the lower left corner.

It is also apparent from the figures that the Pareto front for CGP-NASV1 tends to be more disperse in the objective function space. CGP-NASV2 demonstrates a tendency to focus on a more restricted area of the objective function space, typically yielding to improve initial solutions when compared to CGP-NASV1 (see Fig. 13).

In comparison with other methods, particularly those designed by humans, our proposal demonstrates superior performance in terms of both classification error and number of parameters in both datasets. When compared with single-objective methods, our proposal outperforms them in terms of classification error while showing a significant reduction in the number of parameters. It is important to note, however, that minimizing the number of parameters is not the primary objective of our proposal. When comparing with multi-objective methods in terms of classification error, the EEEA-NET [12] method shows better performance, albeit at the cost of a higher number of parameters. In other metrics, our proposal outperforms the other methods presented. Our proposal aims to find solutions with a favorable trade-off between classification error and MAdds, resulting in architectures with reduced parameters and MAdds. In comparison with other proposals that use CGP as a method for architectures representation, such as CGP-CNN [50], Torabi [32], EvoApprox-Nas [13] and LF-MOGP [38], our proposal demonstrates superior performance.

Works like EvoApproxNAS [13] focuses on optimizing values at the hardware level with a multi-objective approach, keeping the CGP representation intact and adding some block functions like bottleneck and residual inverted to the function set while keeping the CGP properties unchanged by only using mutations. On the other hand, none of these proposals modifies CGP at the representation level, instead attacking specific problems, for example by proposing new operators (Torabi [32]) or mechanisms

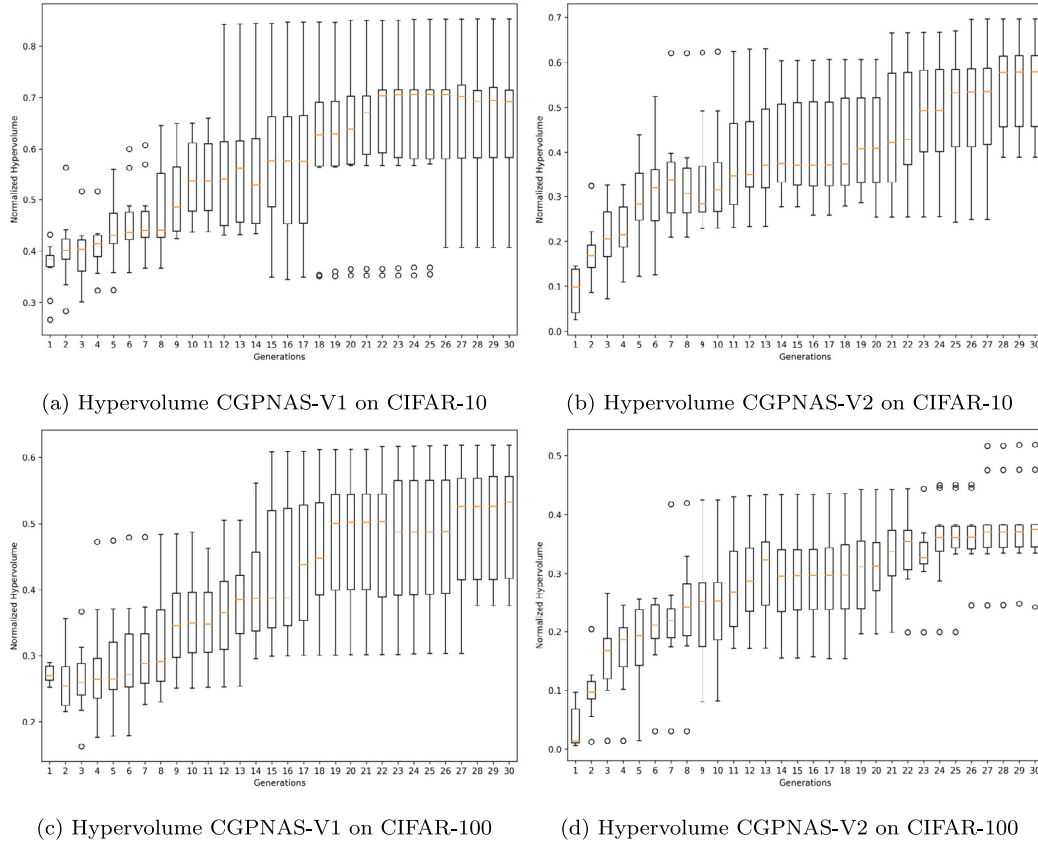


Fig. 12. Box plot of normalized Hypervolume on CIFAR-10 and CIFAR-100.

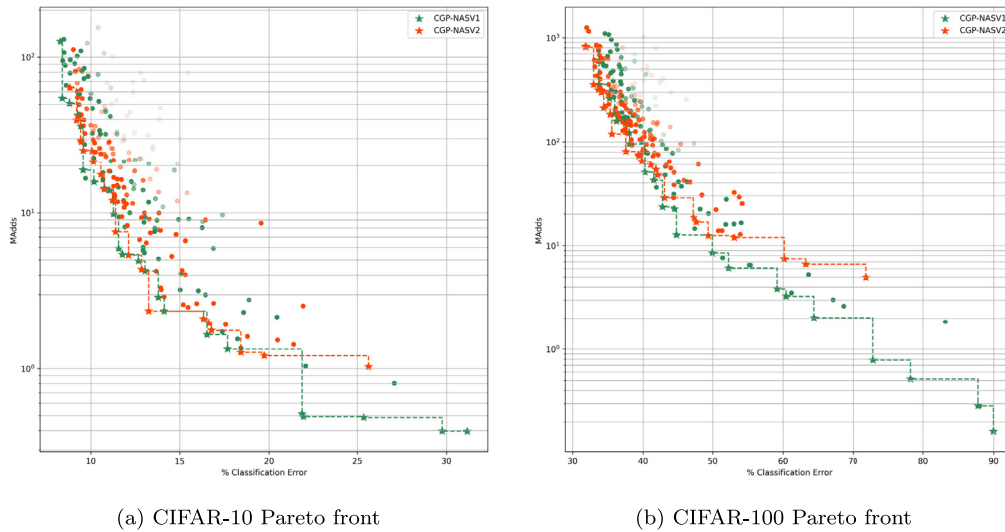


Fig. 13. Comparison of the pareto fronts between CGP-NASV1 and CGP-NASV2, and the population through generations.

to improve the search process (LF-MOGP [38]), which seems to be an important component since the results shown improve compared to other proposals, even though CGP-NASV2 shows a superior performance with canonical MOEAs.

To demonstrate that CGP-NASV1 and CGP-NASV2 are applicable to other image classification tasks, we test them in the SVHN dataset, which are reported in Table 8. When compared to the human-designed methods, CGP-NASV1 and CGP-NASV2 can locate solutions with a classification error quite close to these proposals, standing out in solutions with a smaller number of parameters. It is observed that our results surpass the

multi-objective proposal EvoApproxNAS [13] and single objective Bakhshi's [33] proposal in terms of classification error, number of parameters, and complexity. Our approach, especially CGP-NASV2 where the parameters are included in the search, show a better performance with respect to the number of parameters as well as MAdds, but without affecting in a negative way the error rate.

CGP is known to be a robust method for architecture representation, and the use of real-based representation in our proposal improves the performance, which can be attributed to the relaxation of the search space and the better use of MOEAs, which are designed to operate in a continuous domain.

Table 8

Comparison on SVHN dataset: Classification error rate, the number of parameters and Multiply-adds (MAdds) are expressed in millions (1×10^6), GPU-days and GPU Hardware.

Model	Error rate %	Params	MAdds	GPU-Days	GPU hardware
Human Design					
FractalNet (Larsson et al. 2016) [53]	2.01	38.6	–	–	
Wide ResNet (Zagoruyko et al. 2016) [54]	1.64	2.7	–	–	
ResNet (<i>depth</i> = 101) (He et al. 2016) [48]	2.01	1.7	–	–	
DenseNet (<i>k</i> = 24) (Huang et al. 2017) [47]	1.72	15.3	–	–	
Single Objective Approaches					
(Bakhshi et al. 2020) [33]	4.43	19	–	6	
Multi-Objective Approaches					
EvoApproxNAS (Pinos et al. 2022) [13]	3.09	0.90	247.3	8.8	NVIDIA Tesla V100-SXM2
CGP-NASV2-Best solution	2.70 (2.87 ± 0.15)	2.21 (4.26 ± 1.88)	399.52 (697.71 ± 210.51)	16.25	Nvidia 1080Ti
CGP-NASV2-Knee solution	2.88 (3.05 ± 0.18)	0.49 (0.51 ± 0.16)	55.93 (49.31 ± 11.26)	16.25	Nvidia 1080Ti
CGP-NASV1-Best solution	2.98 (3.18 ± 0.19)	4.18 (5.22 ± 1.91)	729.87 (822.60 ± 300.62)	14.66	Nvidia 1080Ti
CGP-NASV1-Knee solution	2.76 (3.59 ± 0.51)	0.37 (0.29 ± 0.09)	60.34 (34.00 ± 13.64)	14.66	Nvidia 1080Ti

Table 9

Comparison of CGP-NAS versions on the CIFAR-100 dataset, considering both methods for selecting solutions from the Pareto front. Classification error rate, the parameters and MAdds are expressed in millions (1×10^6)

	Knee			Best		
	Error rate (%)	Parameters	MAdds	Error rate (%)	Parameters	MAdds
CGP-NASV2	24.75	1.09	86.49	21.02	5.99	960.01
NSGA-II-DE	(27.15 ± 1.85)	(0.78 ± 0.21)	(71.06 ± 12.03)	(22.66 ± 0.99)	(5.89 ± 2.75)	(1164.75 ± 559.59)
CGP-NASV2	29.47	0.32	36.71	21.12	5.30	1021.62
MOEA/D	(32.46 ± 1.42)	(0.34 ± 0.07)	(30.75 ± 7.70)	(23.88 ± 1.7)	(3.98 ± 1.11)	(601.18 ± 186.12)
CGP-NASV2	23.57	0.49	66.66	20.63	5.99	827
NSGA-II-SBX	(28.43 ± 2.20)	(0.53 ± 0.13)	(55.66 ± 19.14)	(22.49 ± 1.04)	(6.50 ± 1.7)	(850.74 ± 476.08)
CGP-NASV2	25.38	0.95	98.74	21.55	12.19	1320.17
SMS-EMOA	(26.84 ± 1.68)	(0.54 ± 0.19)	(61.09 ± 17.84)	(22.82 ± 1.06)	(7.10 ± 3.21)	(846.15 ± 261.03)

In summary, CGP-NASV1 and CGP-NASV2 demonstrate the ability to identify solutions with a favorable trade-off between two objectives, specifically by achieving a lower number of MAdds when compared to other methods. It is important to note that depending on the specific application or task at hand, different methods for selecting solutions, such as those presented in this work, may be utilized to determine the most appropriate solution for the given scenario.

6.4. Use of alternative MOEAs

To further test the effectiveness of the proposed representation, three additional experiments were conducted. In the first experiment, the differential evolution crossover operator (DE) instead of SBX in NSGA-II was implemented. In a second and third experiment, MOEA/D and SMS-EMOA were implemented as the searching techniques. Using MOEA/D and SMS-EMOA as the optimization algorithms would show the adaptability of the proposed CGP-based solution representation to different evolutionary searching strategies.

The experiments were run 10 times on the CIFAR-100 dataset. The DE operator was configured with CR = 1 and F = 0.5, while MOEA/D and SMS-EMOA used the same crossover and mutation mechanisms of NSGA-II and the configurable parameter to define the number of neighborhoods was set equal 4. The rest of the parameters were configured the same as those listed in Table 3.

Table 9 shows the results for CGP-NASV2 with different evolutionary strategies after 10 executions on the CIFAR-100 dataset. Our experiments revealed several interesting behaviors. In the canonical version of CGP-NASV2 with NSGA-II and SBX, we consistently obtained the best classification errors. When using the

DE operator, we saw similar results to CGP-NASV2, with a slightly lower average and standard deviation, indicating more consistent behavior. Finally, MOEA/D performed similarly to DE while producing solutions with significantly lower complexity, potentially due to its decomposition approach. In the case of SMS-EMOA, we see a very similar performance to NSGA-II-SBX. However, we can notice that on average, in SMS-EMOA, the solutions have a lower or equal error than NSGA-II-SBX. This may be due to the indicator-based approach of SMS-EMOA, but the best solution located is worse than our baseline (see Fig. 14).

To further illustrate these findings, we present the Pareto fronts for each method below, including the average Pareto front for the 10 runs per method. These graphs demonstrate the evolution of the trade-off between classification error and complexity over the course of several generations (see Fig. 15).

One striking feature of the Pareto fronts is the strong similarity between Figs. 14(a) and 14(b), which use the same MOEA. As expected, we see that the DE operator leads to more complex solutions, while MOEA/D exhibits a decrease in MAdds in the early generations, possibly due to the fast convergence of the algorithm, while still maintaining the trade-off in later generations. On the other hand, SMS-EMOA also shares quite a large similarity with NSGA-II-SBX, although we see better distributed solutions across generations because, in the background, SMS-EMOA keeps the solutions that improve the hypervolume.

Overall, the results show the impact of the different mechanisms offered by each MOEA and the impact they have on the trade-off between classification error and complexity in the multi-objective neural architecture search.

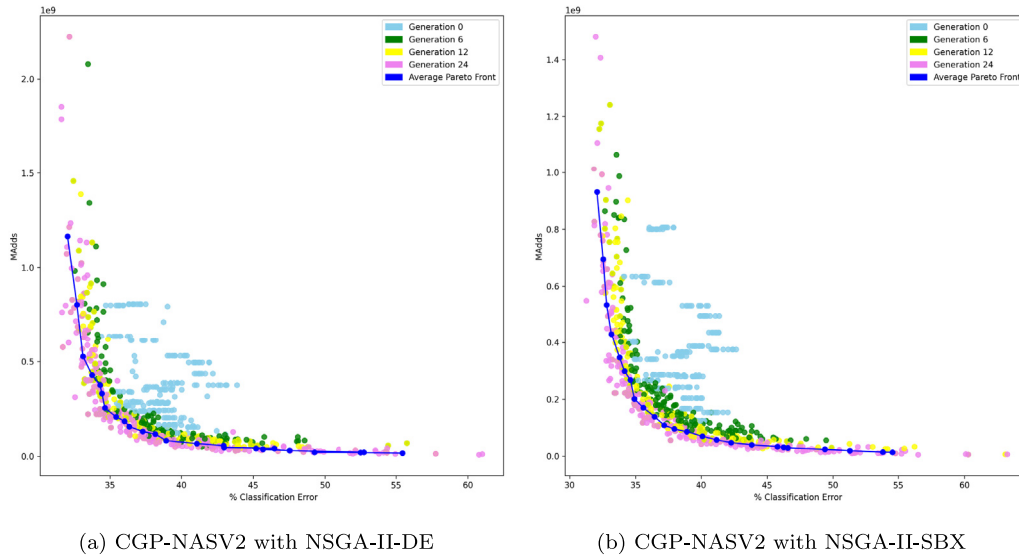


Fig. 14. Evolution progress of CGP-NASV2-SBX and CGP-NASV2-DE on the CIFAR100 dataset, each color refers to a specific generation.

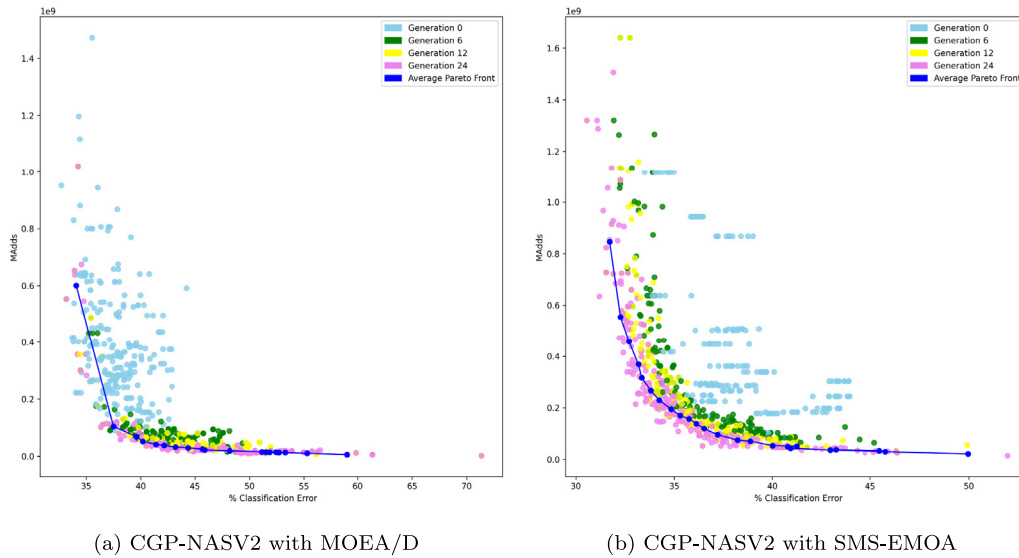


Fig. 15. Evolution progress of CGP-NASV2 with SMS-EMOA and MOEA/D as the core algorithms on the CIFAR100 dataset, each color refers to a specific generation.

6.5. Discussion

Fig. 16 show two CGP-NASV2 evolved architectures selected through the knee and boundary method. Fig. 16(a) correspond to the CIFAR-10 dataset; it is observed that the architecture has more branches together with parameter-free blocks such as summation and concatenation blocks. Fig. 16(b) for the CIFAR-100 dataset, shows an architecture with a more linear structure but with more complex blocks such as MBConv and SepConv. After these observations it is possible to assume that CGP-NASV2 on the more difficult CIFAR-100 data set, evolved by selecting the necessary blocks (even if they are more complex) without significantly increasing the number of parameters or MAdds and therefore maintaining a low classification error. Comparing the best proposed performing NAS algorithm, CGP-NASV2 versus the previous proposal CGP-NAS [11] using the same selection method, there was an improvement from 4.86% to 3.70% in the classification error. However, there is also an increase in the number of parameters and MAdds, that can be reduced by selecting an evolved architecture with the best trade-off between

objectives. Thus a solution with similar performance but with much lower complexity is also available. Similar analysis applies to those evolved architectures for the CIFAR-100 dataset.

7. Conclusion

This study presents a multi-objective evolutionary approach for neural architecture search (NAS) applied to image classification tasks using the CIFAR-10 and CIFAR-100 datasets. The proposed methods, CGP-NASV1 and CGP-NASV2, were able to find CNN architectures that balance classification error and complexity, as measured by the number of MAdds. A key advantage of CGP is its flexibility to explore different evolutionary searching strategies and operators.

Moreover, using the block-chained representation in combination with the continuous CGP offers several advantages for Evolutionary Algorithms (EAs) in the context of Neural Architecture Search. This constraint effectively reduces the dimensionality of the search space, providing greater control over the connections between layers within the architecture. This simplified approach

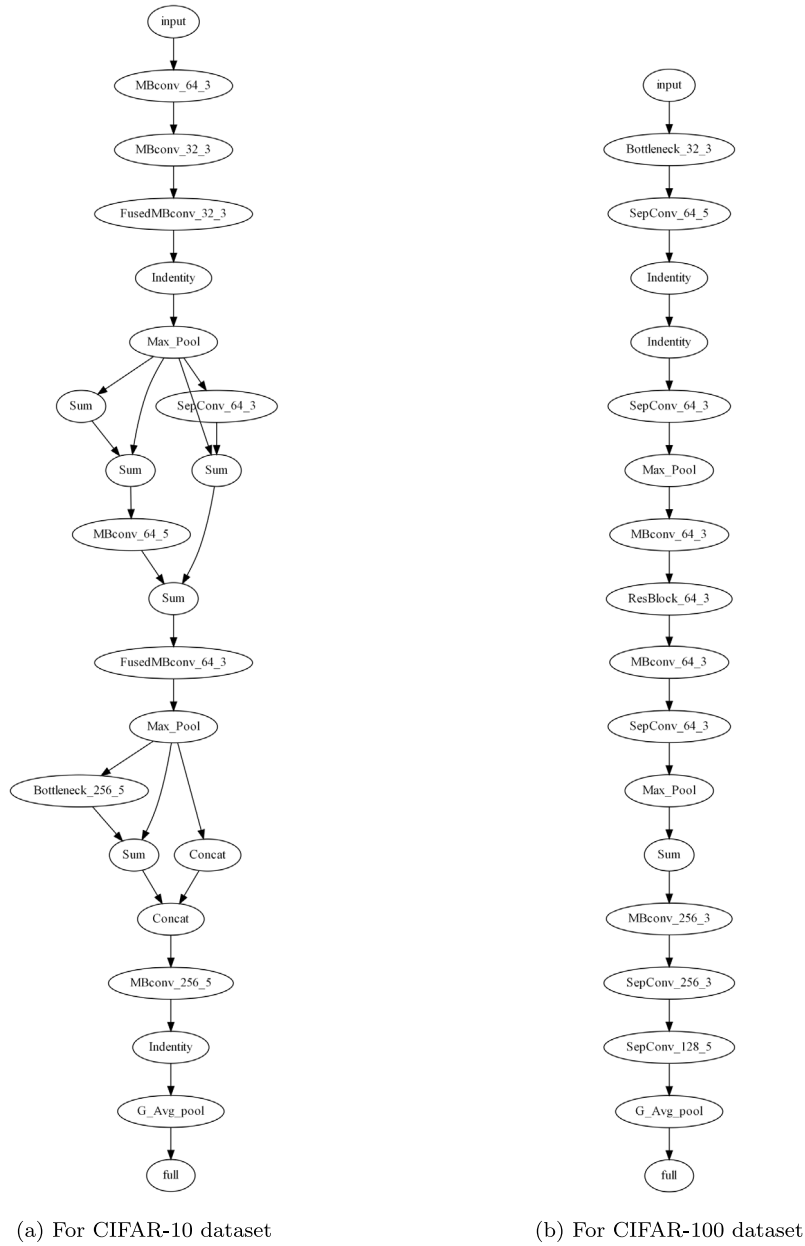


Fig. 16. CGP-NASV2 evolved CNN architectures selected by the knee and boundary method.

has been widely embraced in NAS research [6,7], leading to improved efficiency in the optimization process. Furthermore, our experimental results demonstrate that the block-chained constraint yields competitive outcomes, surpassing the performance of our previous proposal [11], where this constraint was not employed. The inclusion of block-chain encoding has significantly enhanced the effectiveness of our multi-objective NAS variants, aligning them more closely with state-of-the-art approaches in the field.

The results demonstrate that the CGP-NASV2 method is effective in finding CNN architectures with fewer parameters and MAdds compared to other state-of-the-art methods. Additionally, we conducted an exploratory study to investigate the effects of using different multiobjective evolutionary searching strategies and operations. By replacing the crossover operator with the DE operator and using other MOEAs like MOEA/D and SMS-EMOA, we note that our CGP-based representation brings multiple benefits, from allowing variable-length architectures to the adaptation

of multiple operators and search schemes due to the fact that our representation works in a real domain. In summary, the study demonstrates the effectiveness and flexibility of the CGP-based representation for NAS.

For future work, we aim to improve the proposed approach by incorporating additional objectives into the search process, such as latency and model parameters, to better capture the trade-offs between different performance metrics. Also, adding some local search algorithms to learn the weights of the architecture can improve our proposal. Furthermore, we intend to explore the use of surrogate models as a means to speed up the search process while maintaining the quality of the discovered architectures. Lastly, we expect to apply CGP-NASV2 to real-world challenges, particularly in scenarios where architecture complexity plays a critical role, such as mobile or embedded devices. By tailoring our approach to these specific contexts, we aim to offer more efficient and effective neural network architectures for practical applications.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The authors thankfully acknowledge computer resources, technical advice and support provided by Laboratorio Nacional de Supercómputo del Sureste de México (LNS), a member of the CONACYT national laboratories, with project No. 202103083C. This work was supported by CONACYT, Mexico under grant CB-S-26314.

References

- [1] T. Young, D. Hazarika, S. Poria, E. Cambria, Recent trends in deep learning based natural language processing [review article], *IEEE Comput. Intell. Mag.* 13 (3) (2018) 55–75, <http://dx.doi.org/10.1109/MCI.2018.2840738>.
- [2] M. Kolbk, Z.-H. Tan, J. Jensen, M. Kolbk, Z.-H. Tan, J. Jensen, Speech intelligibility potential of general and specialized deep neural network based speech enhancement systems, *IEEE/ACM Trans. Audio, Speech and Lang. Proc.* 25 (1) (2017) 153–167, <http://dx.doi.org/10.1109/TASLP.2016.2628641>.
- [3] S. Grigorescu, B. Trasnea, T. Cocias, G. Macesanu, A survey of deep learning techniques for autonomous driving, *J. Field Robotics* 37 (3) (2020) 362–386.
- [4] A.D. Martinez, J. Del Ser, E. Villar-Rodríguez, E. Osaba, J. Poyatos, S. Tabik, D. Molina, F. Herrera, Lights and shadows in Evolutionary Deep Learning: Taxonomy, critical methodological analysis, cases of study, learned lessons, recommendations and challenges, *Inf. Fusion* 67 (2021) 161–194, <http://dx.doi.org/10.1016/j.inffus.2020.10.014>, [arXiv:2008.03620](https://arxiv.org/abs/2008.03620).
- [5] R. Mikkilainen, J. Liang, E. Meyerson, A. Rawal, D. Fink, O. Francon, B. Raju, H. Shahrzad, A. Navruzyan, N. Duffy, B. Hodjat, Evolving deep neural networks, in: *Artificial Intelligence in the Age of Neural Networks and Brain Computing*, Elsevier, 2019, pp. 293–312, <http://dx.doi.org/10.1016/B978-0-12-815480-9.00015-3>, URL: <https://linkinghub.elsevier.com/retrieve/pii/B9780128154809000153>.
- [6] T. Elsken, J.H. Metzen, F. Hutter, Neural architecture search: A survey, *J. Mach. Learn. Res.* 20 (2019) 1–21, URL: <http://jmlr.org/papers/v20/18-598.html>.
- [7] Z. Lu, I. Whalen, Y. Dhebar, K. Deb, E. Goodman, W. Banzhaf, V.N. Boddeti, Multi-objective evolutionary design of deep convolutional neural networks for image classification, *IEEE Trans. Evol. Comput.* (2020) 1, <http://dx.doi.org/10.1109/TEVC.2020.3024708>, [arXiv:1912.01369](https://arxiv.org/abs/1912.01369), URL: <https://ieeexplore.ieee.org/document/9201169/>.
- [8] A. Eiben, J. Smith, Introduction to Evolutionary Computing, in: *Natural Computing Series*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2015, <http://dx.doi.org/10.1007/978-3-662-44874-8>, URL: <http://link.springer.com/10.1007/978-3-662-44874-8>.
- [9] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, W. Banzhaf, NSGA-Net, in: *Proceedings of the Genetic and Evolutionary Computation Conference*, ACM, New York, NY, USA, 2019, pp. 419–427, <http://dx.doi.org/10.1145/3321707.3321729>, URL: <https://dl.acm.org/doi/10.1145/3321707.3321729>.
- [10] Z. Lu, K. Deb, E. Goodman, W. Banzhaf, V.N. Boddeti, NSGANetV2: Evolutionary Multi-objective Surrogate-Assisted Neural Architecture Search, in: *LNCS*, Vol. 12346 LNCS, 2020, pp. 35–51, <http://dx.doi.org/10.1007/978-3-030-58452-8>, [arXiv:2007.10396](https://arxiv.org/abs/2007.10396).
- [11] C. Garcia-Garcia, H.J. Escalante, A. Morales-Reyes, CGP-NAS, in: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, Vol. 1, ACM, New York, NY, USA, 2022, pp. 643–646, <http://dx.doi.org/10.1145/3520304.3528963>, URL: <https://dl.acm.org/doi/10.1145/3520304.3528963>.
- [12] C. Termitthikun, Y. Jamtsho, J. Ieamsaard, P. Muneesawang, I. Lee, EEEA-Net: An early exit evolutionary neural architecture search, *Eng. Appl. Artif. Intell.* 104 (2021) 104397, <http://dx.doi.org/10.1016/j.engappai.2021.104397>.
- [13] M. Pinos, V. Mrazek, L. Sekanina, Evolutionary approximation and neural architecture search, *Genet. Program. Evol. Mach.* (2022) <http://dx.doi.org/10.1007/s10710-022-09441-z>, URL: <https://link.springer.com/10.1007/s10710-022-09441-z>.
- [14] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002).
- [15] Q. Zhang, H. Li, MOEA/D: A multiobjective evolutionary algorithm based on decomposition, *IEEE Trans. Evol. Comput.* 11 (6) (2007) 712–731, <http://dx.doi.org/10.1109/TEVC.2007.892759>.
- [16] N. Beume, B. Naujoks, M. Emmerich, SMS-EMOA: Multiobjective selection based on dominated hypervolume, *European J. Oper. Res.* 181 (3) (2007) 1653–1669, <http://dx.doi.org/10.1016/j.ejor.2006.08.008>.
- [17] J. Miller, P. Thomson, T. Fogarty, I. Ntroduction, Designing electronic circuits using evolutionary algorithms. Arithmetic circuits: A case study, in: *Genetic Algorithms and Evolution Strategies in Engineering and Computer Science*, 1999.
- [18] J.F. Miller, Cartesian genetic programming, in: *Natural Computing Series*, Vol. 43, 2011, pp. 17–34, <http://dx.doi.org/10.1007/978-3-642-17310>, URL: <http://link.springer.com/10.1007/978-3-642-17310>.
- [19] Y. Sun, G.G. Yen, M. Zhang, Evolutionary Deep Neural Architecture Search: Fundamentals, Methods, and Recent Advances, in: *Studies in Computational Intelligence*, vol. 1070, Springer International Publishing, Cham, 2023, <http://dx.doi.org/10.1007/978-3-031-16868-0>, URL: <https://link.springer.com/10.1007/978-3-031-08020-3>, <https://link.springer.com/10.1007/978-3-031-16868-0>.
- [20] G.A. Vargas-Hakim, E. Mezura-Montes, H.-G. Acosta-Mesa, A review on convolutional neural networks encodings for neuroevolution, *IEEE Trans. Evol. Comput.* (2021) 1, <http://dx.doi.org/10.1109/TEVC.2021.3088631>, URL: <https://ieeexplore.ieee.org/document/9452087/>.
- [21] B. Zoph, Q.V. Le, Neural architecture search with reinforcement learning, 2016, [arXiv preprint arXiv:1611.01578](https://arxiv.org/abs/1611.01578).
- [22] J. Cui, P. Chen, R. Li, S. Liu, X. Shen, J. Jia, Fast and practical neural architecture search, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 6509–6518.
- [23] H. Cai, L. Zhu, S. Han, Proxylessnas: Direct neural architecture search on target task and hardware, 2018, [arXiv preprint arXiv:1812.00332](https://arxiv.org/abs/1812.00332).
- [24] H. Liu, K. Simonyan, Y. Yang, Darts: Differentiable architecture search, 2018, [arXiv preprint arXiv:1806.09055](https://arxiv.org/abs/1806.09055).
- [25] X. Chen, L. Xie, J. Wu, Q. Tian, Progressive differentiable architecture search: Bridging the depth gap between search and evaluation, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, ICCV, 2019.
- [26] X. Chu, T. Zhou, B. Zhang, J. Li, Fair DARTS: Eliminating unfair advantages in differentiable architecture search, in: A. Vedaldi, H. Bischof, T. Brox, J.-M. Frahm (Eds.), *Computer Vision, ECCV 2020*, Springer International Publishing, Cham, 2020, pp. 465–480.
- [27] L. Xie, A. Yuille, Genetic CNN, in: *2017 IEEE International Conference on Computer Vision, ICCV, IEEE*, 2017, pp. 1388–1397, <http://dx.doi.org/10.1109/ICCV.2017.154>, URL: <http://ieeexplore.ieee.org/document/8237416/>.
- [28] M. Suganuma, S. Shirakawa, T. Nagao, A genetic programming approach to designing convolutional neural network architectures, in: *Proceedings of the Genetic and Evolutionary Computation Conference*, ACM, New York, NY, USA, 2017, pp. 497–504, <http://dx.doi.org/10.1145/3071178.3071229>, URL: <https://dl.acm.org/doi/10.1145/3071178.3071229>.
- [29] Y. Sun, H. Wang, B. Xue, Y. Jin, G.G. Yen, M. Zhang, Surrogate-assisted evolutionary deep learning using an end-to-end random forest-based performance predictor, *IEEE Trans. Evol. Comput.* 24 (2) (2020) 350–364, <http://dx.doi.org/10.1109/TEVC.2019.2924461>, URL: <https://ieeexplore.ieee.org/document/8744404/>.
- [30] R.H.R. Lima, D. Magalhães, A. Pozo, A. Mendiburu, R. Santana, A grammar-based GP approach applied to the design of deep neural networks, *Genet. Program. Evol. Mach.* 23 (3) (2022) 427–452, <http://dx.doi.org/10.1007/s10710-022-09432-0>, DOI: 10.1007/s10710-022-09432-0 <https://link.springer.com/10.1007/s10710-022-09432-0>.
- [31] F. Assunção, N. Lourenço, P. Machado, B. Ribeiro, Fast DENSER: Efficient deep NeuroEvolution, in: *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Vol. 11451 LNCS, Springer Verlag, 2019, pp. 197–212, <http://dx.doi.org/10.1007/978-3-030-16670-0>, URL: <https://link.springer.com/chapter/10.1007/978-3-030-16670-0> <http://link.springer.com/10.1007/978-3-030-16670-0>.
- [32] A. Torabi, A. Sharifi, M. Teshnehlab, Using cartesian genetic programming approach with new crossover technique to design convolutional neural networks, *Neural Process. Lett.* (2022) <http://dx.doi.org/10.1007/s11063-022-11093-0>.
- [33] A. Bakhshi, S. Chalup, N. Noman, Fast evolution of CNN architecture for image classification, in: H. Iba, N. Noman (Eds.), *Deep Neural Evolution: Deep Learning with Evolutionary Computation*, Springer Singapore, Singapore, 2020, pp. 209–229, <http://dx.doi.org/10.1007/978-981-15-3685-4>.
- [34] Y.-H. Kim, B. Reddy, S. Yun, NEMO: Neuro-evolution with multiobjective optimization of deep neural network for speed and accuracy Chanwon Seo, in: *JMLR: Workshop and Conference Proceedings*, Vol. 1, 2017, pp. 1–8.
- [35] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, Learning transferable architectures for scalable image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8697–8710.

- [36] B. Wang, B. Xue, M. Zhang, Particle swarm optimization for evolving deep convolutional neural networks for image classification: Single- and multi-objective approaches, in: *Natural Computing Series*, Springer, 2020, pp. 155–184, <http://dx.doi.org/10.1007/978-981-15-3685-4>.
- [37] Y. Xue, P. Jiang, F. Neri, J. Liang, A Multi-objective evolutionary approach based on graph-in-graph for neural architecture search of convolutional neural networks, *Int. J. Neural Syst.* 31 (9) (2021) <http://dx.doi.org/10.1142/S0129065721500350>.
- [38] Q. Liu, X. Wang, Y. Wang, X. Song, Evolutionary convolutional neural network for image classification based on multi-objective genetic programming with leader-follower mechanism, *Complex Intell. Syst.* (2022) <http://dx.doi.org/10.1007/s40747-022-00919-y>.
- [39] Z. Lu, G. Sreekumar, E. Goodman, W. Banzhaf, K. Deb, V.N. Boddeti, Neural architecture transfer, *IEEE Trans. Pattern Anal. Mach. Intell.* (2021) 1, <http://dx.doi.org/10.1109/TPAMI.2021.3052758>, URL: <https://ieeexplore.ieee.org/document/9328602/>.
- [40] M. Tan, Q.V. Le, EfficientNetV2: Smaller models and faster training, 2021, <http://arxiv.org/abs/2104.00298>.
- [41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.C. Chen, MobileNetV2: Inverted residuals and linear bottlenecks, in: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520, <http://dx.doi.org/10.1109/CVPR.2018.00474>, [arXiv: 1801.04381](https://arxiv.org/abs/1801.04381), URL: <http://arxiv.org/abs/1801.04381>.
- [42] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *2016 IEEE Conference on Computer Vision and Pattern Recognition*, CVPR, 2016, pp. 770–778, <http://dx.doi.org/10.1109/CVPR.2016.90>.
- [43] J. Clegg, J.A. Walker, J.F. Miller, A new crossover technique for Cartesian genetic programming, in: *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO '07*, ACM Press, New York, New York, USA, 2007, p. 1580, <http://dx.doi.org/10.1145/1276958.1277276>, URL: <http://portal.acm.org/citation.cfm?doid=1276958.1277276>.
- [44] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, Technical Report, 2009.
- [45] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, A. Ng, Reading digits in natural images with unsupervised feature learning, 2011.
- [46] F.E. Fernandes Jr., G.G. Yen, Pruning deep convolutional neural networks architectures with evolution strategy, *Inform. Sci.* 552 (2021) 29–47, <http://dx.doi.org/10.1016/j.ins.2020.11.009>, URL: <https://linkinghub.elsevier.com/retrieve/pii/S0020025520310951>.
- [47] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4700–4708.
- [48] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [49] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2014, *arXiv preprint arXiv:1409.1556*.
- [50] M. Suganuma, M. Kobayashi, S. Shirakawa, T. Nagao, Evolution of deep convolutional neural networks using cartesian genetic programming, *Evol. Comput.* 28 (1) (2020) 141–163, http://dx.doi.org/10.1162/evco_a_00253.
- [51] E. Real, S. Moore, A. Selle, S. Saxena, Y.L. Suematsu, J. Tan, Q. Le, A. Kurakin, Large-Scale Evolution of Image Classifiers, 2017, URL: <http://arxiv.org/abs/1703.01041>.
- [52] K. Deb, *Multi-Objective Optimization using Evolutionary Algorithms*, Wiley, 2001.
- [53] G. Larsson, M. Maire, G. Shakhnarovich, FractalNet: Ultra-deep neural networks without residuals, 2016, *CoRR abs/1605.07648*, URL: <http://arxiv.org/abs/1605.07648>.
- [54] S. Zagoruyko, N. Komodakis, Wide residual networks, 2016, *CoRR abs/1605.07146*, URL: <http://arxiv.org/abs/1605.07146>.